

EEE3097/8/9S Micromouse 2026: Submission 1

Milestone 1 Code & Demo (25% of Course Mark)

1. Objective

Design and implement a closed-loop controller that guides your Micromouse robot to traverse a perfect **1.0m x 1.0m square** on the floor and stop autonomously. Your design must use closed-loop feedback (such as gyroscope yaw heading alignment and quadrature encoder distances) to remain robust against traction loss, slip, and motor asymmetries.

Specific Learning Objectives:

- Configure and read quadrature wheel encoders to track linear distance.
 - Interface with the MPU6050 IMU gyroscope and integrate angular rate to track heading.
 - Formulate a Proportional (P) or PID controller to synchronize wheel velocities and steer the mouse differentially.
 - Implement a finite state machine (FSM) to transition between straight trajectories and spin-in-place turns.
-

2. Step-by-Step Implementation Guide

- **Sensor Calibration:** Write a test script to verify that pushing the mouse forward increases both encoder counts symmetrically, and that manual rotation returns correct yaw scale factors.
 - **Speed Synchronization:** Adjust left/right motor PWM dynamically so both wheels rotate at the same linear velocity.
 - **Heading Correction:** Add integrated gyro yaw to the feedback loop to correct for drift and bumps.
 - **Spin-in-Place:** Implement a precise 90° right turn using differential spin and a settling window.
 - **State Machine:** Sequence: Drive 1m → Turn 90° → Repeat 4x → Stop. The mouse must halt autonomously.
-

3. Deliverables (Gradescope Submission)

Run the packaging command from your repository root:

```
python tools/package_submission.py --task milestone1 --src workspace/task1_square/
```

Upload the resulting `submission_milestone1.zip` to Gradescope. The package must contain:

1. **Your Controller Code:** Python scripts (`main.py` + custom libraries) OR your Simulink model (`.slx`) and generated C code directory (`*_ert_rtw/`).
 2. **Physical Telemetry Log** (`run_log.jsonl`): Extracted from your mouse using `python tools/dump_logs.py`.
 3. **Physical Run Video** (`run_video.mp4`): Starting with a **3-second close-up of your Student Card** followed by the uncut square run.
-

4. Code & Log Check (Anti-Cheat)

- **Code Match Check:** The autograder compiles/hashes your submitted code and verifies that it matches the FNV-1a "hash" checksum in your telemetry header. Mismatches will reject the submission.
 - **Hardware ID Check:** Microcontroller Device UIDs are tracked. Shared logs under different student accounts will trigger a plagiarism audit.
-

5. Evaluation Criteria

- **Part A: Autograded Co-Simulation Trajectory (80%):** Graded proportionally based on the final Euclidean distance error (d_e) from $(0, 0)$ under motor gain imbalances ($\pm 10\%$) and traction slip:
 - **100%:** $d_e \leq 5$ cm
 - **75%:** $5 \text{ cm} < d_e \leq 15$ cm
 - **60%:** $15 \text{ cm} < d_e \leq 30$ cm
- **Part B: Physical Log & Video Audit (20%):** Valid video with student card (10 points) and matched checksum log (10 points).